

Lab 4: OpenMP, and Three Bugs You Have to See

Write your first OpenMP programs, then deliberately reproduce a data race, a critical-section collapse and a false-sharing slowdown. You cannot recognise them later if you have never made them.

Prepared by **Dr. Abhijith Anandakrishnan**

Assistant Professor, Amrita School of AI

DURATION

Two 2-hour blocks

PREREQUISITE

Unit 2 Part I, Exercise 06

SUBMIT

lab04-<ROLLNO>.pdf

MARKS

15

OFFERING

Odd Semester 2026-27

The point of this lab

Three failure modes account for most broken OpenMP code. All three are invisible in the source, and two of them do not even produce a wrong answer. The only reliable way to learn them is to build them on purpose and watch them happen.

BY THE END OF THIS LAB YOU WILL BE ABLE TO

- Write correct parallel regions and worksharing loops with explicit data sharing.
- Reproduce a data race, demonstrate its non-determinism, and fix it.
- Show that a critical section can make a loop slower than serial, and by how much.
- Reproduce false sharing, and demonstrate the fix with a measurement.
- Choose a schedule from the shape of the work.

Part 0: Warm-up (2 marks)

```
#pragma omp parallel
{
    int id = omp_get_thread_num();
    int n = omp_get_num_threads();
    printf("thread %d of %d\n", id, n);
}
```

0.1 Compile **without** `-fopenmp` and run. How many lines? What does `omp_get_num_threads()` return? Explain why this compiles at all.

0.2 Compile with `-fopenmp`. Run at `OMP_NUM_THREADS = 1, 4, 8`. Are the lines in order? Why not, and does it matter?

0.3 Add `#pragma omp for` around a loop inside the region. Explain in one sentence the difference between `parallel` and `parallel for`.

PITFALL · CHECK THIS BEFORE ANYTHING ELSE, ALL SEMESTER

A missing `-fopenmp` compiles silently, runs on one thread, and shows no speedup. When a parallel program is not faster, check the compile line first.

Part 1: A data race, on purpose (4 marks)

```
double sum = 0.0;
#pragma omp parallel for /* deliberately wrong */
for (long i = 0; i < n; i++) sum += a[i];
```

1.1 Run it 20 times at 8 threads with $n = 10\,000\,000$. Record all 20 answers. How many distinct values? How far is the worst from the correct total?

1.2 Now run 20 times with $n = 1000$. How many distinct values now? Explain why a small problem hides the bug, and what that implies about testing.

1.3 Fix it with `reduction(+:sum)`. Verify 20 identical runs.

1.4 Explain, in terms of read-modify-write, exactly which update is lost.

NOTE · WHY THE GRADER RUNS YOUR EXERCISES REPEATEDLY

Part 1.2 is the whole reason. A single test run of concurrent code proves almost nothing, which is why the autograder repeats the tests across several thread counts.

Part 2: The cure that is worse (3 marks)

2.1 Fix the same loop with `#pragma omp critical` instead. Confirm it is now correct.

2.2 Time three versions at 1, 2, 4, 8 threads: serial, `critical`, `reduction`. Tabulate and plot.

2.3 The `critical` version should be **slower than serial**, and get worse with more threads. Explain both facts.

2.4 Replace `critical` with `atomic` and re-measure. Is it faster? Why is it still much slower than a reduction?

Part 3: False sharing (4 marks)

```
double partial[NTHREADS];           /* 8 doubles = one cache line */
#pragma omp parallel
{
    int id = omp_get_thread_num();
    for (long i = id; i < n; i += NTHREADS) partial[id] += a[i];
}
```

3.1 Confirm the answer is **correct**. There is no race here: each thread writes only its own element.

3.2 Time it at 1, 2, 4, 8 threads against the serial version. Plot speedup. You should see it get **worse** as threads are added.

3.3 Confirm the mechanism with counters:

```
perf stat -e cache-misses,cache-references ./prog
```

Compare the miss count with the amount of data actually touched.

3.4 Apply two fixes and measure each:

- pad each entry to a full 64-byte cache line
- accumulate into a thread-local variable and write once at the end

Report the speedup of each. Which is better, and why would you prefer it even if they measured the same?

DEFINITION · WHAT MAKES THIS BUG SPECIAL

No wrong answer, no race, nothing wrong in the source, and it gets worse the more hardware you throw at it. It is found by measurement, not by reading.

Part 4: Choosing a schedule (2 marks)

Take a triangular loop:

```
for (i = 0; i < n; i++)  
    for (j = i; j < n; j++) work(i, j);
```

4.1 Time it at 4 and 8 threads with `static`, `static,1`, `dynamic,16`, and `guided`. Tabulate efficiency.

4.2 Explain why `static` performs as it does, in terms of how much work each thread receives.

4.3 Try `dynamic,1`. It should be worse than `dynamic,16`. Explain.

What to submit

lab04-<ROLLNO>.pdf with your answers, the three plots (Parts 2, 3 and 4), the perf output from 3.3, and all source as text.

MARKS	FOR
2	Part 0
4	Part 1: the race demonstrated, quantified and explained
3	Part 2: the critical-section slowdown measured and explained
4	Part 3: false sharing reproduced, evidenced with counters, and both fixes measured
2	Part 4: schedule comparison with a correct explanation